VIT Agentic AI Workshop Curriculum

Title: Build Your First AI Agent
Speaker: Vedant
Audience: First- to fourth-year IT/CS students with no agentic AI experience
When: Saturday 17 October 2026, 11:00-13:00 IST
Duration: 120 minutes, including a five-minute break and ten-minute Q&A

Teaching goal

Students build a small canteen agent in a new Python file, understand how it uses tools, and see how the same tools fit LangChain, LangGraph and CrewAI. The core achievement is one working agent and one student-added capability. Framework comparison comes after that first result.

Explain an agent as an AI program that can choose a next step and use permitted tools to work toward a goal. Begin with a lunch request students understand. Introduce each technical name after showing the problem it solves. Explain the few Python functions, arguments, lists and dictionaries as they appear. This is an introduction, not a promise of fluency in Python or three frameworks in two hours.

Teaching edition, 16 October: The kit now contains the first-agent lesson, common local MCP server, LangGraph branch and actual CrewAI tasks. The classroom deck has 36 slides with a matching script and live run sheet. Offline mode uses explicitly labelled scripted brains with real framework and tool execution. Live Gemini calls still need a private key and rehearsal; offline checks do not establish live access or quota. The earlier review-focused examples remain advanced references.

Curiosity before vocabulary

Open with: "You have Rs 80 and ten minutes before your next lecture. Can an AI find lunch from our actual canteen menu?"

Use a two-minute role-play: one person asks, one plays the model, and one holds the menu as the tool. The model requests a lookup and the person holding the menu supplies the result. Then reproduce that exchange in code.

At each transition ask a concrete question: How can the program find out what is available? Who executes the Python? What if we also need opening hours? Can another framework use these tools? What if the canteen is closed? Would a second agent help? Let students predict, run and add a capability. A code audit is not the opening activity.

Exact two-hour schedule

Time IST	Minutes	Activity	Student result
11:00-11:10	10	What is an agent? Lunch challenge and role-play	Explain goal, model, tool and result
11:10-11:20	10	Write and call the menu function	Run the tool directly before adding AI
11:20-11:45	25	Build a LangChain agent in a new file	Add model, tool, instruction and invocation in small steps; understand the ReAct-style loop
11:45-11:55	10	Add a second tool	Add opening hours and ask a question needing both tools
11:55-12:00	5	Break	Resume at noon
12:00-12:15	15	Put the same tools behind MCP	Expose, discover and call a tool; connect using a prepared client connection
12:15-12:27	12	LangGraph with one useful branch	Predict the open and closed routes in an instructor demonstration
12:27-12:37	10	A tiny CrewAI team	See two real agents and a sequential handoff using the same tool source
12:37-12:42	5	Choose a suitable design	Choose for three scenarios; optional A2A explanation and orchestrator sketch
12:42-12:50	8	Project habits and one safety lesson	Write a few Markdown instruction lines; see a forbidden tool request refused
12:50-13:00	10	Questions and next steps	Explain the built agent and choose one extension

The 25-minute build includes about 15 minutes of student typing/running, followed by the ten-minute extension. Include a short local MCP call exercise where setup permits. Preserve the build time. If progress is slow, shorten the later demonstrations to diagrams and recorded outputs; move the A2A teaser and orchestrator sketch after class first.

First build in small steps

Use one new learner file, provisionally my_agent.py. Supply the environment/configuration template and tiny fictional dataset in advance. Students write the application logic without typing SDK internals or provisioning cloud resources.

1. Read the four menu records and write find_food(max_price, vegetarian=True). Call it directly. Explain arguments and return values before introducing tool calling.

2. Add the configured Gemini model and register the function as a tool. Its description tells the model when and how to request it.

3. Use LangChain create_agent with a short instruction. Invoke the lunch question. Run after each small addition rather than pasting a finished application.

4. Show the observable exchange: user request, tool request and arguments, actual result, final answer. The model requests; application code executes. A model can request another tool or finish.

5. Explain the underlying loop with short pseudocode: ask model; if finished, stop; otherwise run an allowed tool, return its result and repeat within a limit. LangChain handles message bookkeeping in this build. A manual model-SDK loop is a follow-up example.

6. Students add get_canteen_hours() and ask for both food and hours. Compare with a menu-only request. Observe actual tool use; do not promise an exact order from a live model.

ReAct is a reasoning-and-action pattern. Use "choose an action, act, observe, repeat" as the beginner explanation. Show tool events and results without requesting private chain-of-thought. This is a ReAct-style tool loop, not a reproduction of the original paper's entire setup.

Common data and tools

Keep the fictional menu: available vegetarian Veg sandwich Rs 60, sold-out vegetarian Poha Rs 35, available Chicken roll Rs 75 and vegetarian Dosa Rs 90. The hours fixture is 09:00-16:00, with mock current time 12:30. Label the mock clock. No actual orders or payments occur.

Tool	Purpose
find_food(max_price, vegetarian=True)	Return available matching menu records
get_canteen_hours()	Return fictional hours and open/closed status

First use local functions. Then expose those same implementations through one MCP server program. The LangChain agent, agent inside LangGraph and CrewAI menu scout consume the same tool contract and dataset through their framework's MCP integration. Connection code differs. An MCP server exposes tools; each agent still needs a model and instructions.

Use local stdio for the workshop. A common server means the same server program and data; each client may start its own local instance. Do not add remote hosting or shared campus server management to the activity. Keep the direct-function agent available if MCP setup stalls.

The preceding speaker covers MCP and LangChain. Reuse that context but briefly explain and call our own tool so students can follow even if the earlier material was unclear.

Three small systems

System	Visible behavior	Pattern and framework
Lunch helper	Chooses food/hours tools and replies	One agent with a ReAct-style loop, using LangChain
Canteen open or closed	Checks hours and calls the lunch agent only on the open route	An explicit graph containing an agent, using LangGraph
Menu scout and lunch planner	Scout gathers available options; planner uses the handoff to propose a small group lunch	Sequential multi-agent collaboration, using CrewAI

The graph needs one branch. Explain state as information carried forward, a node as a step, and an edge as a next route. Keep persistence, approval interrupt/resume, retries and a verification engine for follow-up rather than introducing them together.

The crew demonstration must use actual CrewAI agents. A Python script with role labels is a role simulation and must be described that way. A two-agent sequence is a sequential handoff, not a manager choosing workers. Show an orchestrator-worker diagram only if time permits: a coordinator assigns work and collects

results. Its implementation is optional depth.

Ask whether one agent would already be enough for lunch. The toy team illustrates handoffs; it does not prove multiple agents improve the task. More agents introduce additional calls, latency and coordination.

Patterns, frameworks and protocols

Kind	Examples	Beginner explanation
Pattern	ReAct-style loop, sequential handoff, orchestrator-worker, graph workflow	How work and decisions are arranged
Framework	LangChain, LangGraph, CrewAI	Libraries that help implement those arrangements
Protocol	MCP, Agent2Agent (A2A)	A shared way for separate components to communicate

LangGraph can mix code-controlled and model-controlled decisions. CrewAI supports crews and flows. A framework name does not fix the degree of autonomy. The same design can be implemented in more than one framework.

Introduce A2A only after the team example, for at most a minute if time permits: MCP connects applications to tools; A2A standardizes collaboration between independent agent systems. Two agents in one Python application do not automatically implement A2A. Discovery, protocol messages and distributed agent deployment belong after class.

Choosing a framework

• One assistant choosing from a few tools? Start with a simple loop or a LangChain agent.

• Specific steps, branches or approvals must be controlled? Consider LangGraph, including an agent inside the graph where useful.

• Distinct roles need to pass work between agents? Consider a CrewAI crew and compare it with one agent. Its flows can add explicit orchestration.

These are starting points, not exclusive capabilities or a framework ranking. Let students choose for a menu question, an approval-dependent action and a research-and-writing task.

Project habits and safety after the build

Show why files have different jobs: README.md explains running the project, a supported instruction file states coding rules, data/menu.json stores records, and Python implements behavior. Write a heading, three rules and a command together. AGENTS.md, CLAUDE.md and GEMINI.md support depends on the coding client; arbitrary Markdown is not automatically loaded.

Demonstrate one narrow safety lesson: the model requests an unregistered or forbidden tool and the application refuses to execute it. The completed reference needs a tool allowlist, argument checks, loop limit and private API-key configuration. Prompts guide the model; code enforces boundaries. Longer fake-wrap review, injection and failure-case exercises become optional depth.

For one optional AI-assisted addition, describe the intended change, inspect the new function and run it. Do not ask a coding assistant to generate the entire lesson at once.

Room and access plan

Plan for a possible 100-200 students; attendance and support are not confirmed. Individual laptops are preferred, with optional sharing and keyboard turns. Request 4-6 technical volunteers. Teach from the front using common build checkpoints rather than inspecting every screen.

Install dependencies before class. Students need Python 3.11 and one editor; an AI coding editor is optional. LangGraph and CrewAI can run on the instructor laptop, with optional student follow-up. Avoid simultaneous room-wide account creation or three-framework installation.

A real model run needs eligible personal access, a private key and available quota. Test the exact Gemini model before class. Never distribute the instructor key. Free-tier quota is limited access, not fixed cash credits; editor quota is separate. Python tools and local MCP calls can run without a model. A saved live recording or scripted replay can support blocked students; label which one it is. Neither proves a student's own live agent ran.

Provide checkpoint source files for recovery, keeping a new learner file as the normal route. The earlier offline stand-in remains an optional reference, not evidence of autonomous model behavior. Distribute a tested ZIP before class while the repository is private; a private link alone does not grant access.

Rehearsal before class

Run the local setup check and MCP probe, then the private --live agent check if using Gemini. Rehearse the first build from a new learner file and both graph routes. CrewAI is an instructor installation, optional for students. Use slides/Live Run Sheet.md for exact switches. Record a live backup if using a live model, and practise the full session with a timer. The planned slots total 120 minutes; actual teaching pace still needs rehearsal.

Official references

- LangChain agents: model/tool loop and create_agent.
- LangChain MCP adapters: the pinned adapter used by these examples; match code to installed versions.
- LangGraph overview and patterns: workflows, agents and orchestration.
- CrewAI MCP, processes and flows: common tools and agent coordination.
- MCP architecture and A2A versus MCP: tool access and agent communication.
- ReAct paper: original reasoning/action pattern.